

Take-Home Assignment: Plan vs Actual Tracker

Estimated time: 6–8 hours
Role: Full Stack Developer (3–6 years experience)

Overview

Build a small web application where users set monthly spending targets per category, log actual amounts, and view a report comparing plan vs actual with variance — including support for locked periods.

This exercise evaluates time-series data modeling, aggregation logic, reporting UX, and sensible product rules. No financial planning background is required.

Requirements

1. Authentication

- Implement sign up and log in (email + password is sufficient).
- Each user must only see and modify their own data.

2. Categories

Users can create spending categories (e.g. Marketing, Payroll, Tools).

A fixed seed list is acceptable if you document that CRUD is out of scope — but some way to assign categories to plans and actuals is required.

3. Plans (targets)

For each category, users set a **monthly target** amount.

Example: Marketing → January 2026 → \$5,000.

- Users can create and edit targets for **open** (unlocked) months.
- Targets for **locked** periods cannot be edited (see Locking below).

4. Actuals

Users log actual spend with:

Field	Description
Category	Which category
Month	Month the spend belongs to (YYYY-MM)
Amount	Actual amount spent
Note	Optional free text

Alternatively (or additionally), support CSV import:

```
month,category,amount
2026-01,Marketing,4800
2026-01,Payroll,20500
2026-02,Payroll,19800
```

Validate category names and month format on import.

5. Report

Build a report view where the user selects a date range (e.g. Q1 2026) and sees:

Column	Description
Category	Category name
Month	Month (or rows grouped by category × month)
Plan	Target amount
Actual	Logged actual amount
Variance	Actual – Plan (negative = under plan)
Variance %	(Actual – Plan) / Plan × 100

Handle these edge cases and document your choices:

- **Plan = 0:** do not crash or show NaN (e.g. show — or N/A for variance %).
- **Missing actual:** either treat as 0 or show — for Actual / Variance / Variance %. Be consistent.

Include at least one chart: monthly net variance **or** category totals over the range.

6. Locking

Users can **lock** a month (or quarter — your choice, document it):

- Plans and actuals for locked periods become **read-only**.
- API must reject edit attempts with a clear error — not just hide buttons in the UI.

Document your locking granularity (month vs quarter) in the README.

Sample data

Use this for development and testing. Plans and actuals should produce the variances below when a missing actual is treated as 0 :

Month	Category	Plan	Actual	Variance	Variance %
2026-01	Marketing	5,000	4,800	–200	–4.00%
2026-01	Payroll	20,000	20,500	+500	+2.50%
2026-02	Marketing	5,000	—	— or –5,000	— or –100%
2026-02	Payroll	20,000	19,800	–200	–1.00%

Notes:

- The CSV under **Actuals** matches these actual amounts (Marketing Feb is intentionally omitted).
- For Marketing Feb, either treat missing actual as 0 (variance –5,000 / –100%) or show — for Actual, Variance, and Variance %. Document your choice.

Stretch goals (optional)

- **Drill-down** — click a report cell to see the underlying actual entries.
- **Fiscal year** — support a fiscal year selector (calendar year default is fine).
- **Export** — download the report as CSV.

Technical guidelines

- **Stack:** Your choice.
 - **Deployment:** Required — include a live URL to a deployed version of the app.
 - **Tests:** Tests for aggregation logic, variance calculation, and lock enforcement are appreciated.
 - **Performance:** Mention how you'd index or query at scale in your README, even if the dataset is small.
-

Deliverables

Submit a Git repository containing:

1. **Source code** — backend, frontend, and any migrations/seed scripts.
2. **Deployed live URL** — a publicly accessible link to the running app.
3. **README** with:
 - Prerequisites and step-by-step setup.
 - How variance % is calculated when plan is zero.
 - Locking behavior and granularity.
 - How missing actuals are displayed.
 - Assumptions and tradeoffs.
 - What you would improve before production.
 - The deployed URL (also include it in your submission email).

Optional: a short Loom/video walkthrough (5–10 minutes).

What we evaluate

Area	What we look for
Correctness	Report numbers match stored data; variance math is right
Product rules	Lock enforced server-side; missing data handled consistently
Data modeling	Sensible schema for plans, actuals, categories, locks
Reporting UX	Scannable table/chart; date range filtering works
Import	CSV validation if implemented
Communication	README clarity and documented edge cases

Questions?

If anything is ambiguous, make a reasonable assumption, document it in your README, and proceed.

Good luck — we look forward to reviewing your work.